

Pose Detection

Through the human pose estimation model in the MediaPipe machine learning framework, the human body posture is detected. When the robot detects a person falling, it will sound an alarm and sway from side to side.

1. Program Logic

First, import the human pose estimation model from MediaPipe and subscribe to topic messages to obtain real-time footage from the camera.

Then, process the image, such as flipping, to detect human body information in the image. Based on the lines connecting the key points of the human body, calculate the limb height to determine the body movement.

Finally, if "falling" is detected, the robot will sound an alarm and move forwards and backwards.

2. Operation Steps

Note: the input command should be case sensitive, and keywords can be complemented using Tab key.

1) Start the robot, and enter the robot system desktop using VNC.

2) Click-on  to open the command-line terminal.

3) Run the command to disable app auto-start app service.

```
~/stop_ros.sh
```

```
> ~/stop_ros.sh
```

4) Enter the command. Press "Enter" to run the program.

```
ros2 launch example fall_down_detect.launch.py
```



5) If you want to close this game, press the "ESC" key in the image interface to exit the camera image interface.

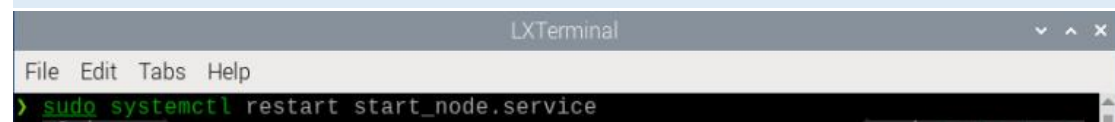
6) Then, press "Ctrl+C" in the command line terminal interface. If it fails, please try again.

After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.



Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

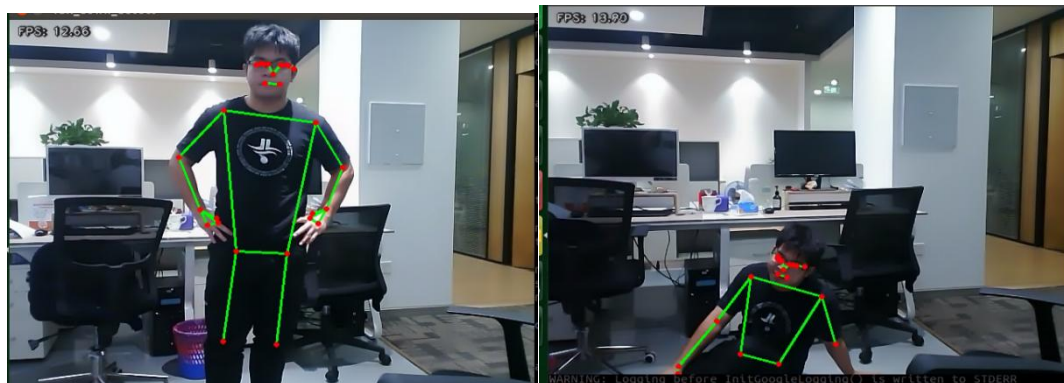
```
sudo systemctl restart start_node.service
```



3. Program Outcome

Once the game starts, ensure the human body remains as fully within the camera's field of view as possible. Upon recognizing the human body, the key points will be highlighted in the returned image.

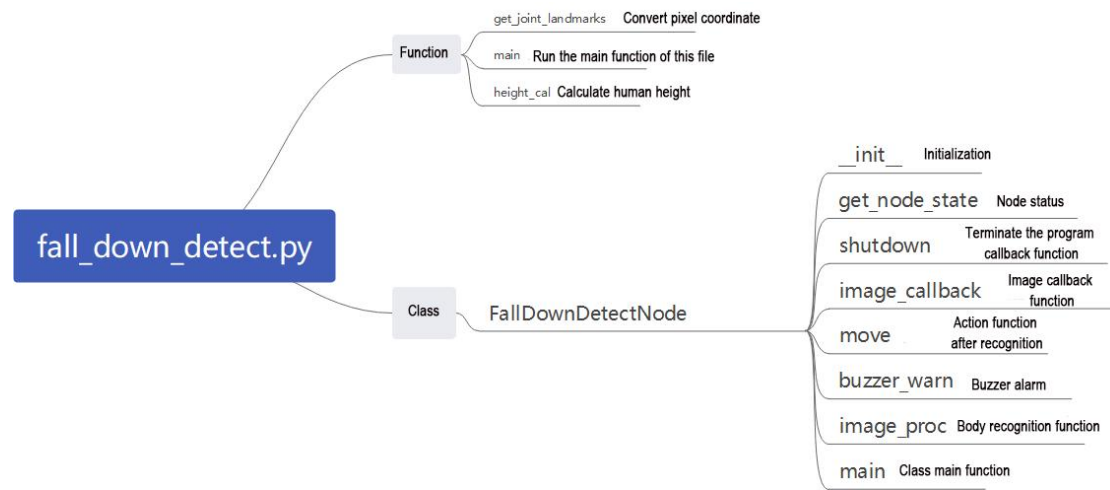
At this point, the individual can sit down briefly. Upon detecting the "falling" posture, the robot will continuously sound an alarm and make repeated forward and backward movements as a reminder.



4. Program Analysis

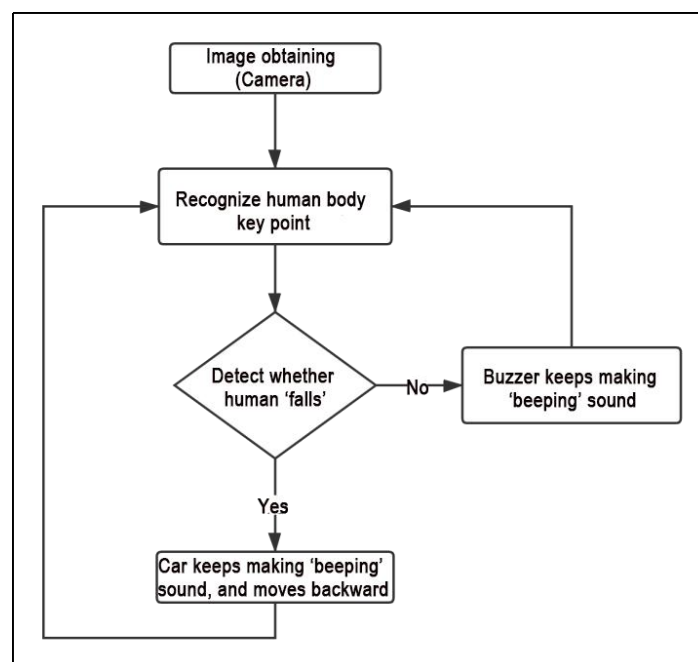
The program file is saved in:

`ros2_ws/src/example/example/body_control/include/fall_down_detect.py`



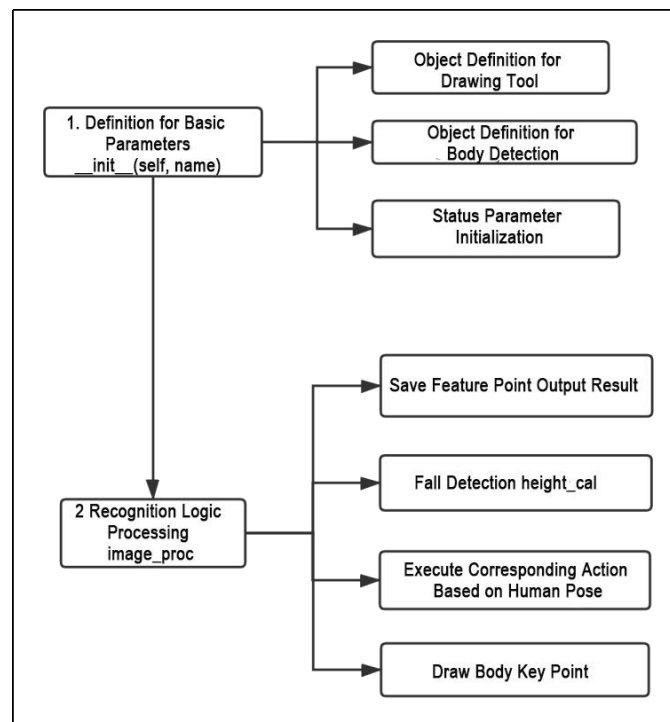
Note: Prior to making any alterations to the program, ensure to create a backup of the original factory program. Modify it only after creating the backup. Directly editing the source code file is prohibited to prevent inadvertent parameter modifications that could render the robot dis-functional and irreparable!

Based on the game's effectiveness, the procedural logic is delineated as follows:



The robot captures images via the camera, identifies the key feature points of the human body, and assesses whether the current posture indicates a "fall". If a fall is detected, the robot's buzzer will emit a continuous "beep" sound while the robot moves backward. Otherwise, the buzzer will only emit a single "beep" sound.

The program logic flow chart obtained from the program files is depicted in the figure below.



4.1 Function

Main:

```

205 def main():
206     node = FallDownDetectNode('fall_down_detect')
207     rclpy.spin(node)
208     node.destroy_node()
  
```

Used to start the body sensation control node.

get_joint_landmarks:

```

43 def get_joint_landmarks(img, landmarks):
44     """
45     将landmarks从medipipe的归一化输出转为像素坐标(Convert landmarks from medipipe's normalized output to pixel coordinates)
46     :param img: 像素坐标对应的图片(picture corresponding to pixel coordinate)
47     :param landmarks: 归一化的关键点(normalized keypoint)
48     :return:
49     """
50     h, w, _ = img.shape
51     landmarks = [(lm.x * w, lm.y * h) for lm in landmarks]
52     return np.array(landmarks)
  
```

Used to convert the recognized information into pixel coordinates.

height_cal:

```
56 def height_cal(landmarks):
57     y = []
58     for i in landmarks:
59         y.append(i[1])
60     height = sum(y)/len(y)
61
62     return height
```

Calculates the height of the limbs based on the recognized information.

4.2 Class

```
62 class FallDownDetectNode(Node):
63     def __init__(self, name):
64         rclpy.init()
65         super().__init__(name, allow undeclared parameters=True, automatically_declare_parameters_from_overrides=True)
66         self.name = name
67         self.drawing = mp.solutions.drawing_utils
68         self.body_detector = mp_pose.Pose(
69             static_image_mode=False,
70             min_tracking_confidence=0.7,
71             min_detection_confidence=0.7)
72         self.running = True
73         self.fps = fps.FPS() # fps计算器(FPS calculator)
74
75         self.fall_down_count = []
76         self.move_finish = True
77         self.stop_flag = False
78         signal.signal(signal.SIGINT, self.shutdown)
79         self.bridge = CvBridge()
80         self.image_queue = queue.Queue(maxsize=2)
```

This class is the fall detection node.

Init:

Initialize the parameters required for body control, read the camera's image callback node, initialize nodes such as chassis, buzzers, and others, and finally start the main function within the class.

get_node_state:

```
99     def get_node_state(self, request, response):
100         response.success = True
101         return response
```

Set the initialization state of the current node.

shutdown:

```
103     def shutdown(self, signum, frame):
104         self.running = False
```

Program exit callback function: used to terminate recognition.

image_callback:


```

100 def image_callback(self, ros_image):
101     cv_image = self.bridge.imgmsg_to_cv2(ros_image, "rgb8")
102     rgb_image = np.array(cv_image, dtype=np.uint8)
103     if self.image_queue.full():
104         # 如果队列已满, 丢弃最旧的图像(if the queue is full, discard the oldest image)
105         self.image_queue.get()
106     # 将图像放入队列(put the image into the queue)
107     self.image_queue.put(rgb_image)

```

Image node callback function: used to process images and enqueue them.

Move:

```

115 def move(self):
116     for i in range(5):
117         twist = Twist()
118         twist.linear.x = 0.2
119         self.mecanum_pub.publish(twist)
120         time.sleep(0.2)
121         twist = Twist()
122         twist.linear.x = -0.2
123         self.mecanum_pub.publish(twist)
124         time.sleep(0.2)
125     self.mecanum_pub.publish(Twist())
126     self.stop_flag = True
127     self.move_finish = True

```

Movement strategy function that moves the robot based on the recognized limb height.

buzzer_warn:

```

129 def buzzer_warn(self):
130     if not self.stop_flag:
131         while not self.stop_flag:
132             msg = BuzzerState()
133             msg.freq = 1000
134             msg.on_time = 0.1
135             msg.off_time = 0.1
136             msg.repeat = 1
137             self.buzzer_pub.publish(msg)
138             time.sleep(0.2)
139         else:
140             msg = BuzzerState()
141             msg.freq = 1900
142             msg.on_time = 0.2
143             msg.off_time = 0.01
144             msg.repeat = 1
145             self.buzzer_pub.publish(msg)

```

Buzzer control function: used for buzzer alarms.

image_proc:

```

147 def image_proc(self, image):
148     image_flip = cv2.flip(cv2.cvtColor(image, cv2.COLOR_RGB2BGR), 1)
149     results = self.body_detector.process(image)
150     if results is not None and results.pose_landmarks:
151         if self.move_finish:
152             landmarks = get_joint_landmarks(image, results.pose_landmarks.landmark)
153             h = height_cal(landmarks)
154             if h > image.shape[:2][0] - 120:
155                 self.fall_down_count.append(1)
156             else:
157                 self.fall_down_count.append(0)
158             if len(self.fall_down_count) == 3:
159                 count = sum(self.fall_down_count)
160
161                 self.fall_down_count = []
162             if self.stop_flag:
163                 if count <= 1:
164                     self.buzzer_warn()
165                     self.stop_flag = False

```

Function for recognizing limbs, which invokes the model to draw key points of the human body based on the recognized information, and moves according to the recognized height.

Main:

```

175 def main(self):
176     while self.running:
177         try:
178             image = self.image_queue.get(block=True, timeout=1)
179         except queue.Empty:
180             if not self.running:
181                 break
182             else:
183                 continue
184
185         try:
186             result_image = self.image_proc(np.copy(image))
187         except BaseException as e:
188             self.get_logger().info('\033[1;32m%s\033[0m' % e)
189             result_image = cv2.flip(cv2.cvtColor(image, cv2.COLOR_RGB2BGR), 1)
190             self.fps.update()
191             result_image = self.fps.show_fps(result_image)
192             cv2.imshow(self.name, result_image)
193             key = cv2.waitKey(1)
194             if key == ord('q') or key == 27: # 按q或者esc退出(press Q or Esc to exit)
195                 self.mecanum_pub.publish(Twist())
196                 self.running = False

```

The main function within the “FallDownDetectNode” class: used to input image information into the recognition function and display the returned image.